

Block Round

プロジェクトの数学

円・楕円・球・楕円体ジェネレータの数学的基礎を述べた技術・教育文書。*Minecraft* のブロックテクスチャでレンダリングされる。各概念について、形式的定義、幾何学的根拠、プロジェクトのソースコードとの直接的対応、ゲーム内構築コマンド（`/fill`、`/clone`、`//sphere`）への実践的翻訳を提示する。

扱う分野：解析幾何・離散ラスタライズ・数字位相・積分学・線形代数・三角法・インベントリ算術・テクスチャマッピング・八面体対称性。

目次

1	概要：問題の数学的性質	3
2	座標系と離散ボクセル格子	4
3	基本方程式：円と楕円	4
4	ユークリッドアルゴリズム（距離テスト）	5
5	Bresenham アルゴリズム（整数中点）	6
5.1	円の場合	6
5.2	楕円の場合（2つの領域）	7
6	しきい値アルゴリズム（角点カバレッジ）	7
7	レンダリングモード：充填、細、太	8
7.1	充填	8
7.2	細（輪郭）	8
7.3	太	9
8	離散面積の計算	9
9	2D から 3D へ：楕円体方程式	10
10	ボクセル化と体積計算	10
11	幾何切断：平面と 45° 対角切断	11
12	太 3D モード：スライスによる三次元化	12
13	3D カメラ：球面座標	12
14	オートズーム：包囲球と FOV	13
15	シェーディングとテクスチャ乗算	13
16	移行：連続数学からゲーム内構築へ	14
17	Minecraft インベントリの算術	15
17.1	単一スロット、単一スタック、満杯のパック	15
17.2	ストレージ：チェスト、ラージチェスト、シュルカーボックス	15
17.3	参照表：標準的な球	15

18 ハイライトオーバーレイと露出面のカウント	16
18.1 何が露出面とみなされるか	16
18.2 境界検出公式	16
18.3 サバイバルヒューリスティックとしての露出面数	16
19 層ごとの建造（水平分解）	17
20 八面体対称性による最適化 (O_h)	17
20.1 具体的なコマンド列	18
20.2 時間コストの比較	18
21 ブロックテクスチャと視覚的構成	19
21.1 6つの面と UV マッピング	19
21.2 ブロック族と色調	19
21.3 ブロック混合によるグラデーション	20
22 結論	20
付録 A：建造リファレンス表と詳細例	22

1. 概要：問題の数学的性質

Block Round は *Minecraft* のブロックテクスチャでレンダリングされた丸い形状のジェネレータで、2D では円と楕円、3D では球と楕円体を扱う。中心となる問題は 離散ラスタライズのクラスに属する：実数 $\mathbb{R}$ 上で解析的に定義された曲線または曲面が与えられたとき、規則的なグリッド（2D ではピクセル、3D ではボクセル）のどのセルをマークすればよいかを決定する。Block Round では、マークされた各セルに加えて *Minecraft* パレットからの 6 面テクスチャ（草ブロック、丸石、オークの板材、クォーツ、ダイヤモンドなど）が付与され、結果として得られる図形は単なる数学的シルエットではなく、ユーザーがサバイバルモードで再現でき、クリエイティブモードで `/fill` を用いて構築でき、Sponge Schematic v2 (`.schem`) 経由で WorldEdit、Litematica、MCEdit にインポートできる建設可能な構造物となる。

この問題には一意の解がない。異なる包含基準が異なる離散シルエットを生み、それぞれの基準が独自の数学的根拠を持つ。そのため、本プロジェクトは 3 つの異なる 2D アルゴリズム（ユークリッド、Bresenham、しきい値）、3 つのレンダリングモード（充填、細、太）、3 つの 3D シェーディングスタイルを実装する。アルゴリズムの選択は視覚的滑らかさだけでなく、ユーザーが集める必要のあるブロック数によっても決まる。直径 $D = 20$ の球はユークリッドアルゴリズムで 4189 ブロックを要するが、しきい値では 4322 ブロックに達する。

実行は完全にブラウザ内で行われ、サーバーコンポーネントは存在しないため、数学的定式化に追加の 計算効率要件が課される（ $D = 32$ のダイヤモンド球は 17156 個のテクスチャ付きボクセルを含めて 60 fps でレンダリングされなければならない）。動員される理論分野は次のとおり：

- 円錐曲線と二次曲面の解析幾何；
- 整数算術による増分アルゴリズム（Bresenham）；
- 数字位相（4-、6-、26-連結近傍）；
- 積分学と計数測度；
- 線形代数（切断平面、透視投影）；
- 三角法と球面座標；
- 組み合わせインベントリ算術と群対称性。

姉妹プロジェクト Pixel Round に対する追加点。 Block Round は、レンダリングされるセルが *Minecraft* のブロックとなる場合にのみ生じる 5 つの数学的関心事項を追加する：(i) 各ボクセルの 6 面の UV マッピング；(ii) サバイバルインベントリの算術（64 個スタック、27 スロットの単一チェスト、54 スロット

のラージチェスト)；(iii) 建設中に露出面を数えるためのハイライトオーバーレイ；(iv) 現実の建設は階ごとに進むため、3D 形状を水平層に分解；(v) 八面体対称群 O_h を利用して `/clone` 経由で手動ブロック配置コストを 8 倍削減する。

2. 座標系と離散ボクセル格子

キャンバスは 2D では $G_x \times G_y$ セルの格子であり、3D では $G_x \times G_y \times G_z$ ボクセルに拡張される。各セル (i, j) は単位正方形 $[i, i+1] \times [j, j+1]$ を占有する；3D では各ボクセルが単位立方体を占有する。連続コードにおいて、セルの **中心** は $(i + \frac{1}{2}, j + \frac{1}{2})$ にある。この約束は重要である：円周をセルの角 (i, j) と比較するか 中心と比較するかで、どのセルが形状に属すると見なされるか、すなわちユーザーがどのブロックを配置しなければならないかが変わる。

$$\text{セル } (i, j) \text{ の中心} = (i + \frac{1}{2}, j + \frac{1}{2})$$

直径 D の形状について、コードは格子を各軸 $D+2$ セルに拡張する（各辺に 1 セルのマージン）。これにより極端なピクセル（N、S、E、W）が行列の境界に落ちることがない。形状の **中心** は $(G_x/2, G_y/2)$ にあり、半軸は $r_x = D/2$ および $r_y = D/2$ である。3D でも同じロジックが z 軸方向に拡張される。キャンバスの単位セルは Minecraft の単位ブロックと同一なので、アルゴリズムの座標はゲーム内 (x, y, z) 座標に一对一でマッピングされる。

Minecraft へのマッピング。 プレイヤーが $(0, 64, 0)$ にスポーンし、地面に乗った $D = 16$ ($r = 8$) の球を望む場合、ワールド座標での中心は $(0, 72, 0)$ （地表から 8 ブロック上）にすべきで、球の南極が地面に接する。WorldEdit コマンド：`(0, 72, 0)` に立って `//sphere stone 8` を実行する。

3. 基本方程式：円と楕円

プロジェクトの全理論は、原点に中心を置き半軸を r_x 、 r_y とする楕円の陰関数方程式から始まる：

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 = 1$$

$r_x = r_y = r$ のとき、楕円は半径 r の **円周** に退化する。内部の点は ≤ 1 を満たし、外部の点は > 1 を満たす。これがボクセルが形状に属するか否か、すなわちユーザーがそこにブロックを配置するか否かを決定する不等式である。

中心を (c_x, c_y) に移し、各セルの中心で評価すると：

$$d_x = \frac{i + \frac{1}{2} - c_x}{r_x}, \quad d_y = \frac{j + \frac{1}{2} - c_y}{r_y}, \quad \text{内側} \iff d_x^2 + d_y^2 \leq 1$$

Minecraft へのマッピング。サバイバルで塔を建てる場合、ユーザーは基礎には幅広く平らな円を、塔が上がるにつれて徐々に小さい円を望むのが普通である。各階は同じ半軸で異なる z を持つ陰関数方程式の適用である。Block Round は 2D 層を一度計算し、ユーザーは `/clone` で各階を複製する。

4. ユークリッドアルゴリズム（距離テスト）

着想。各行 j について、楕円内部にある列 i を解析的に求める。 y を与えたときに x について解く：

$$x = c_x \pm r_x \sqrt{1 - \left(\frac{y - c_y}{r_y}\right)^2}$$

垂直変位 $d_y = j + \frac{1}{2} - c_y$ の行 j について、その高さにおける楕円の 水平半幅は：

$$dxM = r_x \sqrt{1 - \frac{d_y^2}{r_y^2}}$$

$d_y^2/r_y^2 \geq 1$ ならば行は楕円と交わず、スキップされる。さもなければ $c_x - dxM \leq i + \frac{1}{2} \leq c_x + dxM$ を満たすすべての列 i が塗りつぶされる。整数境界は次から得られる：

$$lo = \lceil c_x - dxM - \frac{1}{2} \rceil, \quad hi = \lfloor c_x + dxM - \frac{1}{2} \rfloor$$

根拠。項 $+\frac{1}{2}$ は、インデックス i のセルが中心 $i + \frac{1}{2}$ を持つという約束から生じる。関数 $\lceil \cdot \rceil$ と $\lfloor \cdot \rfloor$ は連続区間を整数インデックスに変換し、**中心** が楕円内部に属するセルのみを含めることを保証する。この定式化は連続曲線に最も近い離散近似を生み、結果として最も視覚的に滑らかなシルエットを得る。しかし小直径では、結果は他のアルゴリズムに比べてピクセルアート性が薄れる $D=5$ のユークリッド円は 13 ブロックのみだが、同直径のしきい値は 21 ブロックある。

Minecraft へのマッピング。ユークリッド基準は WorldEdit の `//sphere` が生成するものに最も近い。 $D=16$ ($r=8$) の丸石球をユークリッド外殻のみで建てるとき、体積は充填モードで $V \approx 2145$ ブロック、細モード (外殻) で約 804 ブロック 34 スタック (2176) を集めるか 13 スタック (832) を集めるかの差である。

5. Bresenham アルゴリズム (整数中点)

Bresenham アルゴリズムは 1965 年に線分描画用に発表され、後に円に拡張され、Pitteway と Kappel によって任意楕円に一般化された。その際立った性質は浮動小数点演算と平方根抽出を不要にすることである：次のピクセルの決定は **整数の加算と比較** のみを用い、2 つの候補ピクセル間の中点が解析曲線のどちら側にあるかを評価する 決定関数による。Minecraft 建築においては、これが最も認識しやすいピクセルアートの見た目を与えるアルゴリズム コミュニティが 2011 年から手作業で建ててきたのと同じ階段状のシルエットである。

5.1 円の場合

整数半径 R の円について、アルゴリズムは $(x=0, y=R)$ から初期決定パラメータで始まる：

$$d_0 = 1 - R$$

各ステップ (0 から 45° の八分円内) で：

$$\begin{cases} \text{もし } d < 0: & \text{次は}(x+1, y), \quad d += 2x + 3 \\ \text{もし } d \geq 0: & \text{次は}(x+1, y-1), \quad d += 2(x - y) + 5 \end{cases}$$

根拠。関数 d は各反復で、2 つの候補ピクセルオプション間の **中点** で評価され

た円の陰関数方程式の値を近似する：

$$F(x+1, y-\frac{1}{2}) = (x+1)^2 + (y-\frac{1}{2})^2 - R^2$$

d の符号は中点が円の内部にあるか（水平方向にのみ進む）外部にあるか（垂直座標も減らす）を示す。円の 8 つの八分円は二面体群 D_4 の対称性を計算された八分円に適用することで得られる。これはコード内では 8 回の `span(...)` 呼び出しに対応する。結果のトレースは、滑らかな曲線の離散近似に特徴的な階段状パターンを示す。

5.2 楕円の場合（2 つの領域）

半軸 a, b の楕円について、アルゴリズムは第一象限を接線が 45° を成す点で区切られた **2 つの領域** に分ける：

$$\text{領域 I: } 2b^2x < 2a^2y \quad (|dy/dx| < 1), \quad \text{領域 II: そうでなければ}$$

初期決定パラメータは：

$$p_1 = b^2 - a^2b + \frac{a^2}{4},$$

$$p_2 = b^2(x + \frac{1}{2})^2 + a^2(y - 1)^2 - a^2b^2.$$

各反復で、 p は事前計算された増分（4 倍した後はすべて整数）で更新される。結果はピクセルあたりの最小演算：1 回の符号テストと 2 回の加算。

Minecraft へのマッピング。 サバイバル建築者にとって、Bresenham のシルエットは離散ステップが手で数えやすいため、ユークリッドより好まれることが多い。ユーザーは次のブロックが常に まっすぐか 斜めかを知っており、配置動作が一貫する。 $D = 11$ の Bresenham 円は数十の解説書で公開されている標準的な 80 ブロックの系列を持つ。

6. しきい値アルゴリズム（角点カバレッジ）

このアルゴリズムは問う：セルのいずれかの角が楕円内にあるか？各セル (i, j) について、コードは形状の中心に最も近い角を見つける（中心のセル辺への射影）：

$$n_x = \text{clamp}(c_x, i, i+1), \quad n_y = \text{clamp}(c_y, j, j+1)$$

$$\text{内側} \iff \left(\frac{n_x - c_x}{r_x} \right)^2 + \left(\frac{n_y - c_y}{r_y} \right)^2 \leq 0,94$$

値 0,94 は 1,0 よりわずかに低い 経験的しきい値である。その役割は、曲線がセルの 1 辺全体に接する 4 つの基本方位点 (N、S、E、W) で生じる 1 ピクセルの 接線スパイクと呼ばれるアーティファクトを除去することである。

3 つのアルゴリズムの中で、同じ直径 D に対して最大面積のシルエットを生むのはこれである。包含条件が最も非制限的だからである。Minecraft 向けには、より多くのブロックを使う代償として、最も 丸みのある見た目を与えるアルゴリズム $D = 16$ はしきい値で 213 外殻ブロック、Bresenham で 200、ユークリッドで 192。

Minecraft へのマッピング。 建築が遠くから見られる場合 (例えば神殿の上のクォーツドーム)、しきい値が選択肢となる。なぜなら少し大きいシルエットは環境遮蔽の影を通して読みやすいからである。トレードオフ：ユークリッドに対して外殻あたり 5–10% 多いブロック。

7. レンダリングモード：充填、細、太

内部セルに対する行列 $f[j][i] = 1$ を持って、プロジェクトは **離散位相** により 3 つの視覚スタイルを導出する：

7.1 充填

f をそのまま返す。すべての内部セルがブロックになる。これは `//sphere stone 8` が生成するものである。

7.2 細 (輪郭)

セルが細い輪郭に属するのは、充填されており かつ少なくとも 1 つの直交隣接 (N、S、E、W) が形状の 外側にあるとき。論理表記：

$$b(i, j) = f(i, j) \wedge (\neg f(i-1, j) \vee \neg f(i+1, j) \vee \neg f(i, j-1) \vee \neg f(i, j+1))$$

幾何学的に：これは形状の **離散境界** で、位相的境界 $\partial F = F \cap \overline{F^c}$ の離散類推である。Minecraft では `//hsphere` (中空球) に等しい：外殻のみが具体化される。

7.3 太

太モードは境界ピクセルに **対角橋** を加える：水平境界隣接と垂直境界隣接を同時に持つ内部セル。これは細モードが残す 1 ピクセル対角を閉じ、シーンで輪郭が安定して見える必要がある場合に有用である（45° の穴がない） ガラスと氷のドームでは特に重要。

$$\begin{aligned} t(i, j) &= b(i, j) \vee (f(i, j) \wedge h_H \wedge h_V), \\ h_H &= b(i-1, j) \vee b(i+1, j), \\ h_V &= b(i, j-1) \vee b(i, j+1). \end{aligned}$$

Minecraft へのマッピング。 $D = 20$ のガラスドーム（中空）について、太モードは 608 ブロックの気密な外殻を生むのに対し、細モードでは約 510 ブロック 余分な ~ 100 ブロックは対角縫合で、環境光が対角の隙間から漏れるのを防ぐ。

8. 離散面積の計算

area2D 関数は単に f にマークされたセルを **数える**。これは 計数測度で、楕円の指示関数のリーマン積分を近似する：

$$\underbrace{\pi r_x r_y}_{\text{連続面積}} \quad \longleftrightarrow \quad \underbrace{\sum_{j=0}^{G_y-1} \sum_{i=0}^{G_x-1} f(i, j)}_{\text{離散面積}}$$

D が大きいとき、離散面積/連続面積 $\rightarrow 1$ 。 D が小さいときアルゴリズム間の差は可視である：しきい値は過大評価しがちで、ユークリッドは理想面積に近く、Bresenham は両者の中間。

Minecraft へのマッピング。 Block Round UI のブロック数チップはまさにこの数字を表示する： $D = 8$ で 268、 $D = 12$ で 904、 $D = 16$ で 2145。チップは数を N (スタック $\times 64$ + 余り) として表現する。したがって $2145 = 33 \times 64 + 33$ 、すなわち “33 スタックと 33 個” で、これがユーザーが実際に集めなければならないものである。

9. 2D から 3D へ：楕円体方程式

3D では、基本形は (c_x, c_y, c_z) を中心とし半軸 r_x, r_y, r_z を持つ **楕円体** である。その方程式は楕円の自然な一般化：

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1$$

$r_x = r_y = r_z$ のとき **球** に戻る。各ボクセルの中心で評価する：

$$a_\xi = \frac{\xi + \frac{1}{2} - c_\xi}{r_\xi} \quad (\xi \in \{x, y, z\}), \quad \text{内側} \iff a_x^2 + a_y^2 + a_z^2 \leq 1$$

Minecraft へのマッピング。 $W = 20$ 、 $H = 10$ 、 $D = 20$ の楕円体は 扁平な球を生み、地下ドームや球技場の屋根に最適。体積は約 2094 ブロック (33 スタック)。等価な WorldEdit コマンド：`//ellipsoid stone 10 5 10`。

10. ボクセル化と体積計算

楕円体の連続体積は積分学の古典的結果で、円板上の積分により得られる：

$$V = \frac{4}{3} \pi r_x r_y r_z$$

離散版は $D_x \times D_y \times D_z$ の箱の各ボクセルを反復し、 $a_x^2 + a_y^2 + a_z^2 \leq 1$ のときカウンタを増やす。これが voxelVolume アルゴリズム。

シェルと内部。 *filled* モードではプロジェクトは保存集合外に少なくとも 1 つの 6-隣接を持つボクセルのみ (つまり外表面または切断面から見えるボクセル) を表示する。*thin* では完全楕円体の **表面** のみが示される (1 ボクセル厚)。サバイバルでは見える面のみが専用のテクスチャ付きブロックを必要とする。内部ボクセルは資源を節約するために安価な充填材 (ダイヤモンドの代わりに土) にできる。

参照表。以下の表は Minecraft コミュニティで最もよく求められる標準球の直径を要約し、ブロック数 (V)、64 のスタック ($P = \lceil V/64 \rceil$)、 $54 \times 64 = 3456$ スロットのラージチェスト ($DC = \lceil V/3,456 \rceil$) に変換する。建設前の資源収集計画に使用する。

直径	ブロック (V)	スタック (×64)	ラージチェスト	メモ
D=8	268	5	1	小ドーム
D=12	904	15	1	中型球
D=16	2145	34	1	大塔頂
D=20	4189	66	2	球技場屋根
D=24	7238	114	3	大ドーム
D=32	17156	269	5	メガ建築

$D \in \{8, 12, 16, 20, 24, 32\}$ におけるユークリッドアルゴリズムで計算された充填モード体積。細モード（外殻）の値は概ね $V_{\text{shell}} \approx 4\pi r^2$ ブロック厚 1 ボクセル、つまり D に応じて充填体積の 25% から 40% の間。

11. 幾何切断：平面と 45° 対角切断

Block Round は **3 つの平面** で立体を切断できる。各切断はボクセルを破棄する線形不等式である：

X 軸: $x \geq cut \Rightarrow$ 破棄

Y 軸: $y \geq cut \Rightarrow$ 破棄

対角: $x + y \geq cut \Rightarrow$ 破棄

X と Y 平面は座標軸に垂直で、法線ベクトルは $(1, 0, 0)$ と $(0, 1, 0)$ 。対角平面は法線 $(1, 1, 0)/\sqrt{2}$ を持ち、XY 平面で 45° で切る：数学的には $x + y = k$ という平面族。 $D_x \times D_y$ の箱における $x + y$ の最大値は $D_x + D_y$ なので、対角切断スライダの範囲は $D_x + D_y$ 、X と Y はそれぞれ D_x と D_y 。切断は **比例的** である：コードは百分率 $cutPct$ を保存し、限界を次のように再計算する：

$$cut = cutPct \cdot \max_{\text{軸}}$$

これによりある軸での 50% は、ユーザーが軸を切り替えたりサイズを変えたりしても 50% のままになる。

Minecraft へのマッピング。球に対する Y 軸 50% 切断は体積の半分の標準的なドームを生む。 $D = 16$ で： $V_{\text{full}} = 2145$ ブロック、 $V_{\text{dome}} \approx 1095$ ブ

ロック（約 18 スタック）。50% の対角切断は 半ボウル断面を生み、噴水内部や円形劇場の座席によく使われる。

12. 太 3D モード：スライスによる三次元化

3D での *thick* モードでは、プロジェクトは 3 つの軸に沿って **すべてのスライス** に 2D 太アルゴリズムを適用する：各 z について XY 、各 y について XZ 、各 x について YZ 。次に結果の 和集合を取る。幾何学的動機は気密性：シェル厚さを倍にせずにすべての対角を塞ぐ最小ボクセル集合。形式的には、 $S_z(z)$ を高さ z の XY スライス、 T を 2D 太演算子として：

$$thick3D = \bigcup_z T(S_z(z)) \cup \bigcup_y T(S_y(y)) \cup \bigcup_x T(S_x(x))$$

結果は切断によって保存された集合と共通部分を取る。背景理論は **数字位相**：太演算子はシェルの 26-連結性（対角を含む隣接）を保証する。これは Minecraft 建築で有用である。なぜなら対角に隣接するボクセルは面で触れない 光、水、生物が対角の隙間を壁がないかのように通過する。

Minecraft へのマッピング。海底神殿上のサバイバル水中ガラスドームには太モードが必須：対角橋は細モードが残す対角の亀裂から水が漏れ込むのを防ぐ。コスト： $D = 20$ ドームは太モードで約 380 ガラスブロックに対し細モードでは約 320 19% 多い材料と引き換えに完全封止。

13. 3D カメラ：球面座標

3D カメラは距離 r で物体を周回し、2 つの角を持つ θ （方位角、水平面）と φ （極角、垂直軸から）。これは球面座標の **物理的慣習** で、デカルト座標への変換（*three.js* が期待する形式）は：

$$\begin{aligned} x &= r \sin(\varphi) \cos(\theta), \\ y &= r \cos(\varphi), \\ z &= r \sin(\varphi) \sin(\theta). \end{aligned}$$

初期位置： $\theta = \pi/4$ （45°、等角投影視点）と $\varphi = \pi/3$ （北極から 60°、赤道の少し上）。マウสดラッグは θ と φ を直接増分する；ホイール/ピンチは r を増分する。このパラメータ化の単純さが、四元数や明示行列なしで自由回転を可能にしている。

14. オートズーム：包囲球と FOV

ユーザーが図形サイズを変えたりカメラをリセットしたりすると、プロジェクトは物体がどの角度でもビューポートに収まるように **理想距離** を再計算する。考え方は物体を半径 R の球（AABB の対角の半分、軸整列バウンディングボックス）に閉じ込めることである：

$$R = \sqrt{(D_x/2)^2 + (D_y/2)^2 + (D_z/2)^2}$$

球は **回転不変の投影** を持つので、ある向きでビューポートに収まれば、すべての向きで収まる。カメラの垂直 FOV ($fov_V = 35^\circ$ ラジアン) が与えられたとき、半径 R の球を枠に入れるのに必要な距離は：

$$dist_V = \frac{R}{\tan(fov_V/2)}$$

水平 FOV はキャンバスのアスペクト比から導かれる：

$$fov_H = 2 \arctan(\tan(fov_V/2) \cdot aspect)$$

最終距離は $dist_V$ と $dist_H$ の最大値に 1.20（20% の視覚マージン）を掛けたもの。図形が切断されている場合、コードは D_x または D_y を残りのサイズに縮小する。オークの木またはクリーパーのイースターエッグがアクティブな場合、バウンディングボックスは上方に拡張されてエンティティの高さを含める。

15. シェーディングとテクスチャ乗算

3 つの 3D スタイル（*Classic*、*Smooth*、*Blocks*）は、各ボクセルの 3 つの可視面（頂、明側、暗側）に適用される **乗算因子** で異なる。関数 `shadeHex(hex, factor)` は `hex` を (R, G, B) にパースし、各チャンネルを乗算してクランプ $[0, 255]$ 。結果はブロックの素のテクスチャの上に適用される色合いになる：

$$R' = \text{clamp}(R \cdot f), \quad G' = \text{clamp}(G \cdot f), \quad B' = \text{clamp}(B \cdot f)$$

これは理想的なランバート照明関数の線形近似である：

$$I = \max(0, \mathbf{n} \cdot \mathbf{l}) \cdot \quad ,$$

ここで 3 つの因子は各面の法線と固定光方向との角度の余弦に対応する。*Smooth* では因子が近い（似た面）；*Classic* では因子がより異なる（強コントラスト、Minecraft バニラの見た目）。*Blocks* はさらに幾何的なくぼみをボクセルに導入する。各面を内側に一定平行移動して、隣接するボクセルが同じテクスチャを持っていたとしてもボクセル間境界を明らかにする。発光ブロック（グロウストーン、シーランタン、シュルームライト、マグマ、エンチャント obsidian）ではランバート項が定数 $I = emissive_factor \cdot texture$ に置き換えられる。

16. 移行：連続数学からゲーム内構築へ

以上の 15 節は、Block Round が姉妹プロジェクト Pixel Round と共有する 連続から離散へのパイプラインを記述する：楕円の陰関数方程式から結果を構図するカメラまで。残りの 6 節は Block Round に **固有** のものを記述する レンダリングされるセルが抽象的なピクセルではなく *Minecraft* ブロックであるという事実によって導入される追加の数学層。各ブロックは 6 つのテクスチャ付き面、インベントリ内の重み、集めて配置するためのプレイ時間コストを持つ。

これらは装飾的な考慮ではない。ブロックテクスチャでレンダリングする決定は、ユーザーが直面する操作問題を変える：*PNG* をエクスポートする代わりに、ユーザーは シルエットを配置プランに翻訳しなければならない。次節の数学的内容はまさにこの翻訳に関わる：インベントリ算術、面数最適化、層ごとの分解、対称性の利用により N 個のブロック配置のコストを $N/8 + 7$ 回のコマンド呼び出しに減らすこと。

17. Minecraft インベントリの算術

Minecraft プレイヤーは 9×4 の格子として組織された 36 スロットの インベントリにアイテムを携帯する。各スロットは同種のアイテムを最大 64 個まで保持する（一 スタックまたは パック）。プレイヤーは ストレージにもアクセスできる：単一チェスト 27 スロット、ラージチェスト 54 スロット。目標ブロック数を収集計画に翻訳する算術は、したがって 天井除算問題である。

17.1 単一スロット、単一スタック、満杯のパック

スタックはインベントリ会計の基本単位である。各スタックは最大 64 個のアイテムを含む。ブロック数 N からパック（またはスタック）への変換は

17.2 ストレージ：チェスト、ラージチェスト、シュルカーボックス

建物が必要とする ラージチェストの数を計算するには、ブロック数を 54 スロット $\times$ 64 アイテム/スロット = 3456 アイテム/ラージチェストで割る：

パック： $N_{\text{packs}} = \left\lceil \frac{N}{64} \right\rceil$

ラージチェスト： $N_{\text{dc}} = \left\lceil \frac{N}{3456} \right\rceil$

単一チェストは $27 \times 64 = 1728$ アイテムを保持する。シュルカーボックス（携帯可能なコンテナ）は $27 \times 64 = 1728$ アイテムを保持するが、それ自体をチェストスロット 内に格納できるので、54 個のシュルカーボックスで埋めたラージチェストは最大 $54 \times 1728 = 93\,312$ アイテムを保持できる。メガ建造では関連する容量単位は シュルカーで満たされたラージチェストである。

17.3 参照表：標準的な球

以下の表は標準的な充填球の体積を収集計画に変換する。最後の列は各直径の建設文脈を提案する：

直径	V (ブロック)	パック	ストレージ	典型的な建造
D=8	268	5	1 箱	小屋の屋根、井戸ドーム
D=12	904	15	1 箱	望楼の頂
D=16	2145	34	1 箱	村人の鐘、図書館ドーム
D=20	4189	66	2 箱	天文台ドーム
D=24	7238	114	3 箱	神殿ドーム
D=32	17156	269	5 箱	大聖堂ドーム、ワールドボスアリーナ

Block Round の情報チップが表示する スタックと余り表現 $N = q \cdot 64 + r$ は、サバイバルインベントリが部分スタックを表示する方法を反映する：プレイヤーは q 個の満杯スロットと r を示す 1 つのスロットを見る。 $D = 16$ では、チップは “2145 (33 × 64 + 33)” と印字し、プレイヤーに 33 スロットを完全に埋めて 1 スロットに 33 アイテムを入れるように告げる。ちょうど 34 スロットで、インベントリ 1 行に余分な 4 スロット。

18. ハイライトオーバーレイと露出面のカウント

Block Round はデフォルトで各可視ボクセル面の縁に黒い ハイライトオーバーレイをレンダリングする。視覚的にはゲーム内デバッグオーバーレイの “F3+G” チャンク境界スタイルに似ているが、ボクセル単位までスケールされる。数学的にはオーバーレイは **露出面カウント** の可視化を実装する：サバイバル建築者がブロック単位で進捗を検証するのに使う数値。

18.1 何が露出面とみなされるか

ボクセル $\mathbf{v}$ が隣接 $\mathbf{n}$ と共有する面は、 $\mathbf{n} \notin V_{\text{solid}}$ のとき（隣接が空）のときのみ露出している。各固体ボクセルは 0（完全に埋まる）から 6（孤立）の面を寄与する。露出面の合計 F_{exp} は図形の 表面積の離散類推。

18.2 境界検出公式

ボクセルが境界上にあるのは、その 6 つの面隣接の少なくとも 1 つが固体の外側にある場合：

$$b(i, j, k) = f(i, j, k) \wedge (\neg f(i \pm 1, j, k) \vee \neg f(i, j \pm 1, k) \vee \neg f(i, j, k \pm 1))$$

これは 2D 細アルゴリズムの 3D 拡張の同じ境界演算子で、オーバーレイは各境界ボクセルの立方体辺線を単に描画する。ガラスと氷ではデフォルトオフ、丸石、石、ダイヤモンドや他の不透明ブロックではデフォルトオン。

18.3 サバイバルヒューリスティックとしての露出面数

サバイバル建築者にとって、関連するカウントは総体積 V ではなく 露出面数 F_{exp} である。なぜならそれが外から見えるものだからである。すべての固体ボクセルにわたる F_{exp} の合計は、境界ボクセル数の 6 倍引く境界ボクセル間の面隣接数の 2 倍に等しい：

$$F_{\text{exp}} = \sum_{\mathbf{v} \in V_{\text{solid}}} \sum_{\mathbf{n} \in N_6(\mathbf{v})} [\mathbf{v} + \mathbf{n} \notin V_{\text{solid}}]$$

$D = 16$ の球（充填モード）の場合、 $V = 2145$ 、 $V \approx 432$ （外殻）、 $F_{\text{exp}} \approx 1184$ 可視面 建造は約 432 の 見えるブロックだけで必要で、残りの 1713 ブロックは内部で最も安価なブロック（土、丸石）で埋められる。オーバーレイは外殻のブロック欠落を即座に発見できるようにする。

19. 層ごとの建造（水平分解）

Minecraft では建造は 水平層ごとに進む：プレイヤーは層に立ち、上の層のブロックを置き、それからブロック 1 つ分上に上がる（ジャンプ + 一時的足場ブロックを置く、または elytra を使う）。したがって 3D 楕円体問題は $2D$ 楕円のスタックとして最もよく理解される、層ごとに 1 つ：

$$\text{層}(k) = \{(i, j) : a_x(i)^2 + a_y(j)^2 \leq 1 - a_z(k)^2\}$$

各整数層 k について $-r_z$ から $+r_z$ まで、横断面はそれ自身が縮小した半軸 $r_x(k)$ と $r_y(k)$ を持つ楕円：

$$r_x(k) = r_x \sqrt{1 - \left(\frac{k}{r_z}\right)^2}, \quad r_y(k) = r_y \sqrt{1 - \left(\frac{k}{r_z}\right)^2}$$

これは 3D 問題が 前節の $2D$ アルゴリズムを 1 層ずつ計算し、結果を積み重ねることに還元されることを意味する。認知的に膨大な単純化：3 つの座標を心の中で追跡する代わりに、建築者は層あたり 2 つの座標と層インデックスを追跡する。

Minecraft へのマッピング。 $D = 16$ ($r_z = 8$) の球について、赤道層 ($k = 0$) は完全な $D = 16$ Bresenham 円（約 200 ブロック）。次の層 ($k = 1$) は $D = 15,97$ の円。 $k = 4$ では層は $D = 13,86$ の円（約 148 ブロック）、 $k = 7$ では $D = 7,48$ の円（約 43 ブロック）に縮む。極 ($k = \pm 8$) は単一ブロックの帽子。

Block Round の情報チップは、ユーザーが中心ガイドコーナーボタン（キー c）をアクティブにすると 3D モードでこの層別カウントを表示する。水平スライスは赤道と各 $r_z/2$ 分割で半透明黄色十字として描かれる。

20. 八面体対称性による最適化 (O_h)

原点を中心とする球は位数 48 の **八面体群** O_h の作用の下で不変である。球のボクセル化は、3 つの座標反射と各軸まわりの 3 つの $\pi/2$ 回転で生成される位数 48 の部分群を保存する。実用的な建設目的では、関連する部分群は $x \mapsto -x$ 、

$y \mapsto -y$ 、 $z \mapsto -z$ で生成される位数 8 の 八分体反射群 $\mathbb{Z}_2^3$ である。1 つの八分体のボクセル集合が球全体を決定する：

$$V_{\text{total}} \approx 8 V_{\text{octant}} \implies \text{削減} = \frac{N - N/8}{N} = 87,5\%$$

これは Minecraft 建築者が利用できる最大の数学的最適化である。 N 個のブロックを手動配置する代わりに、建築者は正八分体に $N/8$ のブロックを配置し、適切な反射で 7 つの `/clone` (バニラ) または `//copy` + `//paste` (WorldEdit) コマンドを発行する。

20.1 具体的なコマンド列

ワールド座標 (0, 72, 0) を中心とする $D = 24$ ($r = 12$ 、 $V = 7238$ ブロック) の球について、まず正八分体 ($+x, +y, +z$ 領域、約 905 ブロック) を建造し、それから発行する：

```
/clone 0 72 0 12 84 12 ~~~ filtered minecraft:stone
/clone 0 72 0 12 84 12 -12 72 0 mode:masked (-x 反射)
/clone 0 72 0 12 84 12 0 60 0 mode:masked (-y 反射)
/clone 0 72 0 12 84 12 0 72 -12 mode:masked (-z 反射)
/clone -12 72 0 -1 84 12 mode:masked (-x, -y 八分体)
/clone 0 60 -12 12 71 -1 mode:masked (-y, -z 八分体)
/clone -12 72 -12 -1 84 -1 mode:masked (-x, -z 八分体)
/clone -12 60 -12 -1 71 -1 mode:masked (-x, -y, -z 八分体)
```

各 `/clone` は約 50 ms のサーバー時間を消費するので、7 回の反射は半秒未満で完了する。八分体の 905 ブロックの手動配置はサバイバルで約 15 分、クリエイティブのブラシで約 45 秒。7238 ブロックの完全配置 (対称性なし) はサバイバルで約 2 時間、クリエイティブで約 6 分。

20.2 時間コストの比較

T_{block} をブロックごとの配置時間 (典型的にサバイバルで 1–2 秒、クリエイティブで 0,1–0,5 秒)、 T_{clone} をクローンごとのコマンド時間 (約 50 ms) とする。対称性ありとなしの総建設時間は：

$$T_{\text{octant}} + 7 T_{\text{clone}} \ll T_{\text{full}}$$

サバイバルで $N = 7238$ の場合 ($T_{\text{block}} = 2\text{s}$ 、 $T_{\text{clone}} = 0,05\text{s}$)：完全 = 14476 s $\approx$ 4 時間 1 分；八分体 + クローン = 1810 s + 0,35 s $\approx$ 30 分。8 倍の加速で、反射部分群の位数と正確に一致する。位数 48 の完全八面体群 O_h は原理的にさらに削減できる (48 倍) が、対角回転を活用するには 45° 回転した建造領域が必要で、バニラ Minecraft ではセットアップコストを払う価値が稀。

21. ブロックテクスチャと視覚的構成

Minecraft ブロックは平面に着色されたセルではなく、6つのテクスチャ付き面を持つ立方体である。Block Round は各ボクセルをバニラリソースパックからサンプリングされた 6 つの標準面テクスチャでレンダリングする。数学的にはテクスチャ化操作は UV マッピングである：単位立方体の各面が $[0,1]^2$ 座標でパラメータ化され、対応するテクスチャがその UV でサンプリングされて、レンダリングされたピクセルを生成する。

21.1 6 つの面と UV マッピング

各ボクセル面について、レンダラはブロックモデルを参照する：

$$\text{面} \in \{\text{頂, 側, 底}\} \quad \text{UV} : [0,1]^2 \rightarrow \text{pixels}$$

ほとんどのブロック（石、丸石、ダイヤモンドブロック、オークの板材）は 6 面すべてに同じテクスチャを使う。多面ブロック（草ブロック、カボチャ、干し草の俵、クォーツの柱、すべての木の原木、作業台、かまど、本棚、TNT）は頂、側、底で異なるテクスチャを使う：草ブロックは緑の頂、草混じり土の側、ただの土の底；オークの原木は頂と底に木目のリング、4 つの側面に樹皮を持つ。決定的に、どのボクセルを配置するかは数学は使用するテクスチャに依存しない。同じ直径のダイヤモンド球と丸石球のシルエットは同一である。テクスチャの決定は幾何アルゴリズム完了後に適用される。純粋に審美的な層である。

21.2 ブロック族と色調

利用可能な数十のブロックから選ぶサバイバル建築者にとって、関連する質問はテクスチャインデックスではなく 色調族：明/暗、暖/冷、均一/パターン化。以下の表は Block Round 建造で最もよく使われるブロックを分類する：

ブロック	族	色調	典型的な用途
cobblestone	石	灰	素朴な壁、ダンジョン
stone	石	灰	清潔な壁、台座
oak_planks	木	褐	床、内装仕上げ
quartz_block	クォーツ	白	神殿、現代ドーム
sandstone	砂	褐	砂漠、ピラミッド
deepslate	石	暗	地下、アクセント

21.3 ブロック混合によるグラデーション

単一のブロックタイプは 1 つの色調を提供するが、グラデーションは層ごとに 2 つまたは 3 つのブロックタイプを交互配置することで得られる。古典的な Minecraft 技法はドームの石からクォーツへのグラデーション：下層は滑らかな石、赤道層は滑らかな石と閃緑岩のノイズの混合、上層はクォーツ。各層は同じ Block Round アルゴリズムで計算されたままで、変わるのはボクセルごとのテクスチャ参照。Block Round の *Random* ブロックオプションはデフォルトでそのような手続き的混合を実装する（草の蓋、その下に土、中央に散らばる鉱石、岩盤近くに深層岩）。数学は第 19 節の層分解と同じで、ボクセルごとの“ブロックタグ”だけが変わる。

22. 結論

本文書は Block Round で使われている数学的基礎を体系的に記述した。約 1000 行の JavaScript コードと支援するテクスチャおよび音声パイプラインで、プロジェクトはいくつかの理論分野からの貢献を統合している：

- **解析幾何**: 楕円と楕円体の陰関数方程式を所属の主基準として。
- **古典的ラスタライズアルゴリズム**: 中点定式化での Bresenham、楕円用 Pitteway/Kappel 拡張付き。
- **数字位相**: 離散境界演算子、4-、6-、26-連結性、スライス合成、露出面カウント。
- **積分学**: 楕円体の体積を三重積分として、その離散対応物（ボクセル上の計数測度）。
- **線形代数**: 線形不等式で定義される切断平面と 3 次元透視投影。
- **三角法**: 球面座標でのカメラパラメータ化と FOV に応じた自動距離調整。
- **組み合わせインベントリ算術**: 64 のパックと 3456 アイテムのラージチェストの天井除算会計、情報チップが表示するスタック・余り表現。
- **群対称性**: 八面体群 O_h とその位数 8 の反射部分群 $\mathbb{Z}_2^3$ を利用して `/clone` 経由で建設コストを 8 倍削減。

研究が定式化を許す中心観察は次のとおり：離散格子上では、連続曲線の **唯一正しい表現は存在しない**。本文書で提示された各アルゴリズムはセル包含基準の異なる数学的定義に対応し、各定義は独自の形式的性質を持つシルエットを

生む ユークリッドの滑らかさ、Bresenham の階段パターン、角点カバレッジ基準のより大きいカバレッジ。アルゴリズムの選択はしたがって技術的決定であり、意図された視覚表現、結果形状の望ましい計量特性、そして 特に Block Round では 形状をゲーム内で物理的に建設するためのインベントリと時間コストを考慮しなければならない。

Block Round は Minecraft ツールエコシステムにおいて小さいが数学的に豊かなニッチを占める：純幾何ツール（WorldEdit、Litematica）と純視覚ツール（リソースパック、シェーダー）の間に位置し、連続方程式から離散配置プランへの明示的、導出可能、再現可能な橋を提供する。姉妹プロジェクト Pixel Round は Minecraft 層なしで同じアルゴリズムを提供し、伝統的なピクセルアートとボクセルアートアプリケーションに適している。両プロジェクトは同じ連続から離散への数学的パイプラインがいかに根本的に異なる芸術的および建設的ワークフローを支えるかを例示する。

付録 A：建造リファレンス表と詳細例

本付録は Block Round ユーザーが最もよく使う数値・コマンド参照を集めたものである。データは文書全体に散らばる節別の表と詳細例を統合し、建造計画、コマンド構文、エンドツーエンドの詳細例の一ページ参照として機能するように整理されている。

A.1 球の包括的リファレンス（充填・中空・パック）

以下の表は第 10 節と第 17 節の標準球表を拡張し、中空殻バリエーション（細モード）を充填体積と並べて記載する。中空殻のカウントは 1 ボクセル厚を仮定しユークリッドアルゴリズムで計算される；太モードは第 12 節の説明通り対角隙間を封じるため約 10–15% 多くのブロックを加える。

D	充填 (V)	中空 (殻)	パック充填	パック中空
8	268	170	5	3
10	523	316	9	5
12	904	500	15	8
14	1437	744	23	12
16	2145	1042	34	17
18	3052	1338	48	21
20	4189	1648	66	26
24	7238	2400	114	38
28	11494	3296	180	52
32	17156	4332	269	68

サバイバル建築者にとって 中空列が通常関連する：可視殻のみが専用テクスチャブロックを必要とし、内部（必要なら）は最も安価な材料で埋められる。パック-充填列は完全充填球の総備蓄サイズを示す；パック-中空は中空ドームまたは殻の現実的最小値。

A.2 WorldEdit / バニラコマンドリファレンス

以下の表はゲーム内で Block Round 出力を建造するのに最も関連する WorldEdit およびバニラ Minecraft コマンドを要約する。WorldEdit コマンドはプレイヤーが杖（デフォルト木斧）を持ち領域を選択していることを前提とする；バニラコマンドはモッドなしで動作するが座標引数が必要。

コマンド	モッド / 出所	結果
//sphere stone 8	WorldEdit	充填球 r=8 (ユークリッド)
//hsphere stone 8	WorldEdit	中空球 r=8 (殻のみ)
//cyl stone 8 16	WorldEdit	円柱 r=8, h=16
//ellipsoid st. 10 5 10	WorldEdit	楕円体 10×5×10
/fill ... stone	バニラ	一種類のブロックで箱を埋める
/clone ... mode:masked	バニラ	領域を新しい場所にコピー

Litematica ユーザーの場合ワークフローは異なる：コマンド実行の代わりに、schematic ファイル (Block Round は .schem、Sponge 形式 v2 をエクスポート) を Litematica モッドにロードし、それが図形の半透明な幽霊を表示し、プレイヤーがブロック単位で配置する。これは WorldEdit のクリエイティブ体験に最も近いサバイバルモード等価物。

A.3 詳細例：神殿上の D=20 クォーツドーム

完全な詳細例として、建築者が既存の石神殿の上に $D = 20$ のクォーツドームを欲しているとする。神殿の上部は 11×11 ブロックで、ワールド座標 $(0, 80, 0)$ に中心がある。ドームは半径 $r = 10$ の球を $Y = 0$ 平面で切った上半分。建造計画：

- Block Round で計画。** アプリを開き、3D モード、球、サイズ $D = 20$ 、切断軸 Y を 50% に設定。ピッカーからクォーツブロックを選択。情報チップは $V_{\text{dome}} \approx 2126$ ブロック (約 34 パック、第二ラージチェストに 1330 アイテム残るラージチェスト 1 個) と報告する。
- 資源を集める。** 34 パックのクォーツブロック $= \lceil 2126/4 \rceil = 532$ クォーツブロック $\times 4$ ネザークォーツ各 $= 2128$ ネザークォーツ。適宜精錬または交易する。
- アルゴリズム決定。** 下から見られるクォーツドームの場合、ユークリッドアルゴリズムが最も滑らかなシルエットを与える；遠隔山上のドームならしきい値アルゴリズムがきれいに見える。Block Round はデフォルトでユークリッド。
- ドーム配置。** ドームの平面は神殿の上部と一致しなければならない。バウンディングボックスは $20 \times 10 \times 20$ ブロック；中心を $(0, 80, 0)$ に置けば、平面 (切断) 面が $y = 80$ で頂点が $y = 89$ 。
- 対称で建造。** ドームの $+x, +z$ 象限を建造 (約 530 ブロック)。`/clone 0 80 0 10 89 10 -10 80 0 mode:masked` を発行して x で鏡映；次に `/clone -10 80 0 10 89 10 0 80 -10 mode:masked` で z で鏡映。総経過時間：サバイバルで約 10

分。

6. **仕上げ**。頂点と赤道の四基本方位にシーランタンを追加して発光照明（Block Round はそれらを MC 光レベル 15 の発光マップでレンダリングする）。最後の数ブロックを置く前にドーム殻に隙間がないか確認するために、必要に応じて Block Round のエッジオーバーレイ（キー g）を切り替える。

各ステップの数学はまさに前 22 節が記述するもの：陰関数楕円体方程式、ユークリッドボクセル化、Y 軸切断、`/clone` による八面体対称削減、体積をパックとチェストに翻訳するインベントリ算術。付録は新理論ではなく この文書のすべての理論概念が建造計画ワークフローにおける単一の具体的決定にマップされることを思い出させるもの。

A.4 キーボードショートカット参照

Block Round アプリはキャンバスのコーナーボタンを反映する小さなキーボードショートカット集を露出する。クイックリファレンスのため以下にリストする；ショートカットはグローバル（入力フィールドにフォーカス不要）で、特記なき限り 2D と 3D モードの両方で動作する。

キー	切替	備考
G	エッジグリッドオーバーレイ	各ボクセル面に黒い輪郭
C	中心ガイド	軸に半透明な黄色十字
D	PNG ダウンロード	現在のキャンバスを PNG 画像として
I	情報チップ	ブロック数とパック内訳
M	2D / 3D 切替	平面とボクセルモードを切替
S	音オン/オフ	環境ブロック設置音
T	昼 / 夜テーマ	背景を暗くし、タイルを読みやすく保つ
Ctrl+Z / Y	元に戻す / やり直す	図形変更履歴を辿る

ショートカット g（グリッド）と c（中心ガイド）は建造検証時に特に有用：g はシェルの各ボクセルが配置されているかを示し（欠けブロックはオーバーレイパターンを壊す）、c は切断が図形軸に適切に中心揃えされていることを確認する。

Ctrl+Z の元に戻す：Block Round は図形を変更する各編集（スライダー、レンダリングモード、アルゴリズム、形状、軸、ブロック、モード切替）を辿るが、純粹視覚的切替（カメラ、エッジオーバーレイ、中心十字、テーマ、音）は意図的に除外する。この分離は元に戻すスタックをユーザーが取り消したい編集で埋め、何気ない視覚調整ではない。

A.5 主要用語集

本文書全体に繰り返し現れる技術用語のクイックリファレンス。各エントリは概念を簡潔に要約し、それが詳細に展開される節を指す。

- **ボクセル** — 3D における単位立方体、2D ピクセルの類似物。各 Minecraft ブロックは 1 ボクセル。第 2 節参照。
- **パック (スタック)** — Minecraft において、インベントリ 1 スロットを占める 64 個の同一アイテムのスタック。第 17 節参照。
- **ラージチェスト** — 隣接する 2 つのチェストを統合した 54 スロットのコンテナ。最大 3456 アイテムを保持。第 17 節参照。
- **シュルカーボックス** — チェストスロット内に格納可能な、27 スロットの携帯コンテナ。第 17 節参照。
- **シェル** — 固体図形の外側 1 ボクセル厚層。サバイバル建築者にとって関連する表面。第 7 節と第 18 節参照。
- **八分体** — 3 つの座標半空間の交差により得られる 3D 空間の 8 つの領域の 1 つ。対称性最適化に使用。第 20 節参照。
- **UV マッピング** — 3D 面の 2D パラメータ化、各ボクセル面にテクスチャ画像をマップするために使用。第 21 節参照。
- **Bresenham** — 1965 年の増分ラスタライズアルゴリズム、整数演算のみを使用。第 5 節参照。
- **しきい値** — セルのいずれかの角が楕円内にあればセルを含めるラスタライズ基準。第 6 節参照。
- **ユークリッド** — セルの中心が楕円内にあればセルを含めるラスタライズ基準。第 4 節参照。
- **イースターエッグ** — 特定の入力で起動する隠し機能。Block Round では：オークの木とクリーパー、両方ともスライダー値 15 で。
- **スキマティック** — WorldEdit、Litematica、MCEdit でロード可能な直列化された構造ファイル (Sponge Schematic v2、`.schem`)。

この用語集は完全な数学辞典より意図的に短く保たれている。基礎数学（数字位相、積分学、群論など）のより深い扱いを求める読者は、本文書の関連節を参照されたい。

A.6 モデルの展望と限界

本文書で提示された Block Round モデルは、楕円、楕円体およびそれらの切断の静的ボクセル化と、Minecraft 層によって導入される実用的考慮を扱う。動的問題—移動する図形、アニメーションボクセル遷移、流体シミュレーションとの相互作用—は離散ラスタライズの範囲外の連続時間フレームワークを必要とするため、扱わない。自然な拡張はスキマティックレベルのアニメーション：補間で接続された静的ボクセル化のシーケンスで、Block Round はすでに Litematica スクリプトで読める `.schem` ストリームとしてエクスポートできる。

もう一つの未解決方向は正確な体積保存ボクセル化：現在の離散カウント V_{disc} は連続体積 V_{cont} に漸近的にのみ収束する。正確なカウントが連続式と一致しなければならない応用（例：レンダリング競技、厳格なブロック予算の数学アート）では、追加の洗練層が必要—例えば、離散カウントが目標値に達するように半径をわずかに調整。Block Round は現在この洗練を実装していないが、根底にある数学は単純： $V_{\text{disc}}(r) = V$ を r について数值的に解く。

最後に、高次対称性の活用：本文書は O_h の位数 8 反射部分群を扱うが、完全な位数 48 群は原理的に $\times 48$ 加速を許す回転を含む。実用的な障害は、ゲーム内コマンドが軸整列領域で動作し、回転対称を活用するには 45° 回転した建造領域が必要なこと。選択ボックスを内部的に回転するサーバー mod (FAWE など) は原理的に達成できるが、主流ツーリングはまだ公開していない。バニラサーバーで $\times 48$ 加速を実証する参照実装は、本研究の自然な後続となるだろう。

A.7 追加参照と外部リソース

本文書で展開された数学的概念には、それぞれ確立された文献がある。一次資料やより深いアルゴリズムの背景を求める読者には、以下のエントリポイントが推奨される：

- J. E. Bresenham, *Algorithm for computer control of a digital plotter*, IBM Systems Journal, vol. 4 no. 1, 1965 本文書で拡張された整数演算ラスタライズの原論文。
- M. L. V. Pitteway, *Algorithm for drawing ellipses or hyperbolae with a digital plotter*, Computer Journal, vol. 10, 1967 Bresenham を任意の円錐曲線に一般化。
- A. Kappel, *An ellipse-drawing algorithm for raster displays*, ACM Comm. vol. 28, 1985 第 5 節で使用される 2 領域楕円公式。
- R. Klette と A. Rosenfeld, *Digital Geometry: Geometric Methods for Digital Picture Analysis*, Morgan Kaufmann, 2004 第 7 節と第 18 節の数字位相と境界演算子の標準参照。

- WorldEdit ドキュメント, enginehub.org/worldedit/ 本文書全体で使用する `//sphere`、`//hsphere`、`//ellipsoid`、`//copy` 系コマンドの公式参照。
- Sponge Schematic 仕様 v2, github.com/SpongePowered/Schematic-Specification Block Round が `.schem` としてエクスポートするファイル形式。

第 22 節にリストされた 8 つの理論領域には、それぞれ単一の技術文書が要約できる範囲をはるかに超える教科書レベルの取り扱いがある。Block Round の貢献は理論自体にではなく、具体的な合成にある：通常純粋な幾何ツールから省略されるインベントリとシンメトリーコストに関する明示的考慮を含めて、テクスチャ付きボクセル Minecraft 設定に古典的ラスタライズを適用すること。

再現性に関する最終的な注釈：本文書のパイプライン全体 ビルドスクリプト、翻訳、LaTeX テンプレート、ロケール別表 はプロジェクトリポジトリの `docs_math/` ディレクトリに含まれる。XeLaTeX/MiKTeX を備えたシステム上で `python build.py` を再実行すると、サポートされている 9 つのロケールのいずれでも、この PDF を正確に生成する。姉妹プロジェクト Pixel Round は同じ構造に従い、テクスチャ付きとテクスチャなしバリエーションの直接の側並びの比較を可能にする。

文書終了

Block Round は Vinícius Rodrigues de Souza によって維持されている。姉妹プロジェクト Pixel Round は github.com/ViniSouza128/pixel-round で利用可能；現在の Block Round リポジトリは github.com/ViniSouza128/block-round にある。

コメント、修正、翻訳改善は、プロジェクトリポジトリのイシュートラッカーを通じて歓迎する。8 つの非英語ロケールのネイティブスピーカーによる校正は特に重視される。ローカライズはかなりの検討を経て準備されたが、技術 - 数学用語はネイティブの修正から恩恵を受ける。